

① operaciones conflictivas

→ operacion / minimo item
 ↳ de menos 1 es escritura

1.1)

$$H_1 = r_1(A); w_1(A); r_2(A); w_2(A); r_1(B); w_1(B);$$

$$r_2(B); w_2(B);$$

|||

$$H_2 = r_1(A); w_1(A); r_2(A); r_1(B); w_2(A); w_1(B);$$

$$r_2(B); w_2(B)$$

|||

$$H_3 = r_1(A); w_1(A); r_1(B); r_2(A); w_1(B); w_2(A); r_2(B); w_2(B)$$

|||

$$H_3 = \underbrace{r_1(A); w_1(A); r_1(B); w_1(B)}_{T1}; \underbrace{r_2(A); w_2(A); r_2(B); w_2(B)}_{T2}$$

1.2) RC / NO RC = Historio Recoverable

$$\Leftrightarrow T_i \text{ lee de } T_j \text{ si } i \neq j \text{ en } H$$

$$\forall c_i \in H \Rightarrow c_j < c_i$$

es decir T_i hizo commit despues del commit de aquella que lee.

ACA = Avoids Corcorating Aborts

$$\Leftrightarrow T_i \text{ lee } x \text{ de } T_j \text{ con } i \neq j \Rightarrow c_j < r_i(x)$$

Lee solo valores de x que ya hicieron commit.

ST = strict $\Leftrightarrow$ nunca que

$$w_j(x) < o_i(x) \text{ si } i \neq j$$

$$\Rightarrow w_j < o_i(x) \text{ o } c_j < o_i(A)$$

No se puede leer/escribir un item hasta que la tx que lo escribio previamente haya hecho commit/abort

T_3 lee de T_4

$$H_1 = r_1(x); r_1(y); w_1(x); w_4(x); r_3(x); c_1;$$

$$w_4(y); w_3(y); w_4(z); c_4; w_3(x); c_3$$

• RC ✓

• ACA X

• ST X

• podría ser dirty read
 • podría ser lost update

$$H_2 = r_1(x); w_2(x); r_1(y); w_1(x); w_1(y); c_1; r_2(z); w_2(y); c_2$$

• RC ✓

• ACA ✓

• ST X

• Dirty read
 • podría ser lost update

$$H_3 = w_1(x); w_2(x); w_2(y); c_2; w_1(y); c_1; w_3(x); w_3(y); c_3$$

• RC ✓

• ACA ✓

• ST X

• Dirty read
 • ✓ lost update

$$H_4 = w_2(x); r_1(x); r_2(z); r_1(y); w_2(y); w_1(y); c_2; w_1(x); c_1$$

• RC ✓

• ACA X

• ST X

• Dirty read X
 • lost update ✓

$$H_5 = r_1(x); r_1(y); r_3(x); w_1(x); w_4(y); w_3(x); c_1; w_3(y);$$

$$w_4(z); c_3; w_4(x); c_4$$

• RC ✓

• ACA ✓

• ST X

• Dirty read X
 • lost update ✓

Dirty read = T_i reads from T_j , $w_j(x) < r_i(x)$
 AND $c_j \neq r_i(x)$

Lost update = 2 or more tx update same obj, overwriting the other.

↳ requires READING on history? or is it enough just overwriting?

1.3)

H1: RC, ACA, ST

H2: NoRC, NoACA, NoST

H3: RC, ACA, NoST

1.4)

a) ^{NoRC} $H_1 = w_1(B); r_2(D); r_2(B); r_1(C); w_1(C); w_2(C);$
 $c_2; c_1$

No RC porque T_2 lee de T_1
 $\forall c_2 < c_1$

b) $H_2 = r_2(D); r_2(B); w_1(B); r_1(C); w_1(C); w_2(C);$
 $c_1; c_2$

RC y ACA (porque nadie lee de algo escrito en la historia)
 pero no es ST porque T_1 escribe C y T_2 escribe en C antes de que c_1 commit.

c) $H_3 = w_1(B); r_1(C); w_1(C); c_1; r_2(D); r_2(B); w_2(C); c_2$
 RC, ACA, ST.

② $li(A)$ lockea A, $un(A)$ deslockea A.
2.1)

a) WTF qué sentido tiene lockear
dentro de una misma Tx

b) Historia legal $\rightarrow$ construcción de tx

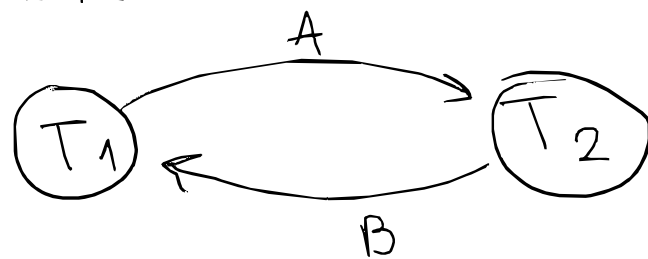
- $\rightarrow$ Ti puede w/R de item x si $li(x)$ y no $mu(x)$
- $\rightarrow$ si $li(x) \Rightarrow mu(x)$

$\rightarrow$ legalidad $\rightarrow$ p/ lockear x, nadie más puede estar lockeando x

Historia serializable $\Rightarrow$ conflicto equivalente a Historia Serial

$T_1 = l_1(A); r_1(A); u_1(A);$
 $l_1(A); w_1(A); u_1(A);$
 $l_1(B); r_1(B); \mu_1(B);$
 $l_1(B); w_1(B); \mu_1(B)$ idem T_2

b) $H = l_1(A); r_1(A); w_1(A); \mu_1(A);$
 $l_2(A); r_2(A); w_2(A); \mu_2(A);$
 $l_2(B); r_2(B); w_2(B); \mu_2(B);$
 $l_1(B); r_1(B); w_1(B); \mu_1(B)$



prova de precedencia
c/ ciclo. $\therefore$ No SR.

2.2) Historia legal c/ dirty read

$T_1 = l_1(A); l_1(B); \mu_1(B); \mu_1(A); l_1(C); \mu_1(C)$

$T_2 = l_2(A); l_2(E); l_2(D); \mu_2(E); \mu_2(A); \mu_2(D)$

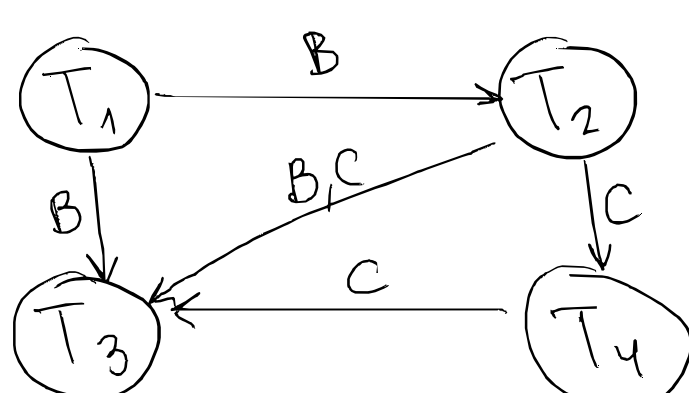
Si hago

$H = T_1; T_2; C_1; C_2$

hace un dirty read x T_2
 está leyendo de A valores
 no commitados escritos x T_1 .

2.3)

a) SR? $SG(H)$ tiene que ser acíclico



es acíclico $\Rightarrow$ es serializable

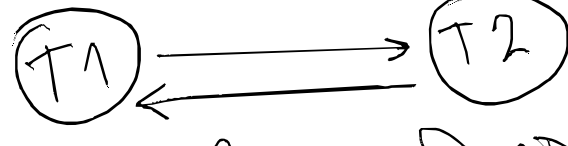
Historia serial Equiv:

$T_1; T_2; T_4; T_3$

2.4) $T_1 = l_1(A); r_1(A); w_1(A);$
 $l_1(B); r_1(B); w_1(B);$
 $\mu_1(A); \mu_1(B)$

2.5) a) T_1 es 2PL
 T_2 es 2PL

b) $l_1(A); l_2(A); l_2(B); l_1(B); \dots$



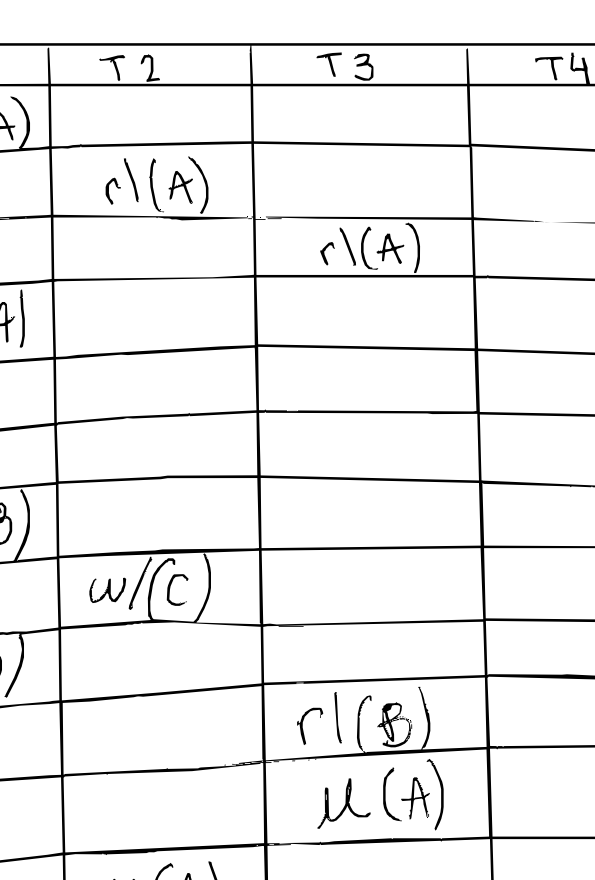
ciclo $\therefore$ DEADLOCK

3.1)

a) $H = w_1(A); A=A+1; w_1(B)=B=A+B; u_1(A);$
 $rl_2(A); u_2(C); C=A+1; u_2(D); D=1; u_2(A);$
 $u_2(D); u_2(C); u_1(B)$

b) Si.

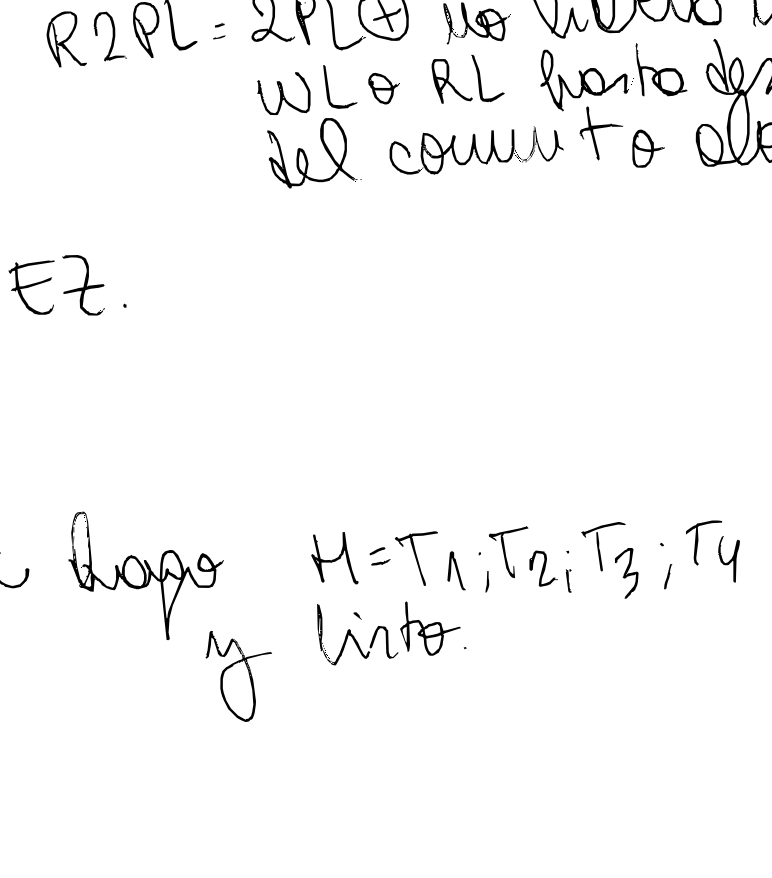
3.2)



SR, $T_2; T_3; T_1; T_4$

3.3)

T_1	T_2	T_3	T_4	T_5
$rl(A)$				
	$rl(A)$			
		$rl(A)$		
$u(A)$				$rl(B)$
				$u(B)$
$w(B)$				
	$w(C)$			
$u(B)$				
		$rl(B)$		
		$u(A)$		
	$u(A)$			
			$w(A)$	
			$u(A)$	
$rl(D)$				
$u(D)$				
			$w(D)$	
			$u(D)$	
				$w(A)$
				$u(A)$
	$u(C)$			
		$u(B)$		



NO SERIALIZABLE

c)

$T_1 = rl_1(A); u_1(A); u_1(B); u_1(B);$ ~~2PL~~
 $rl_1(B); u_1(D);$
 $T_2 = rl_2(A); u_2(C); u_2(A); u_2(C);$ 2PL
 $T_3 = rl_3(A); rl_3(B); u_3(A); u_3(B);$ 2PL
 $T_4 = u_4(A); u_4(A); u_4(D); u_4(D)$ ~~2PL~~
 $T_5 = rl_5(B); u_5(B); u_5(A); u_5(A)$ ~~2PL~~

EZ por los 2PL

d)

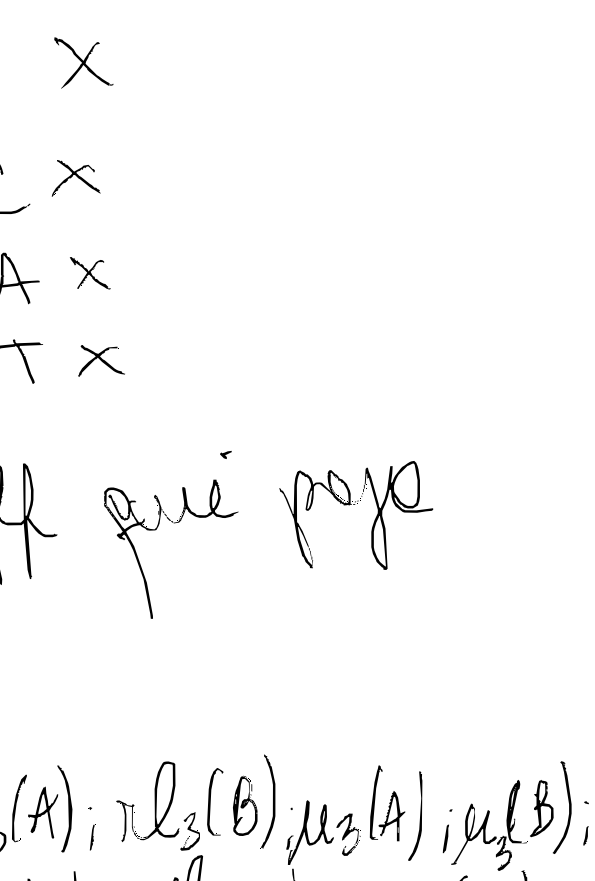
S2PL = 2PL $\oplus$ no libero ningún lock de w hasta desp del commit o abort
R2PL = 2PL $\oplus$ no libero ningún WL o RL hasta desp del commit o abort

EZ.

3.4)

a) $H = T_1; T_2; T_3; T_4$ y listo.

b)



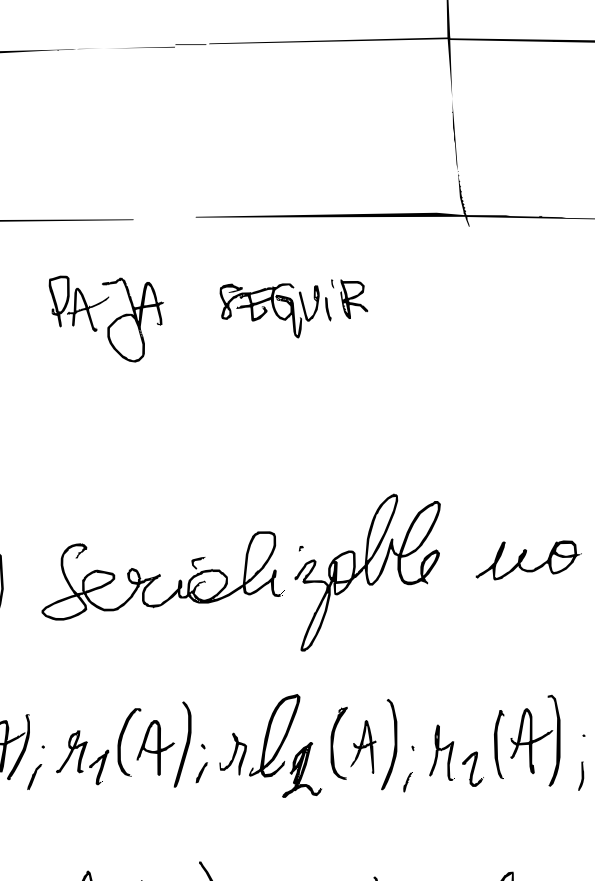
ciclo, NO SR.

c)

EZ.

3.5)

CREO que hay un error...



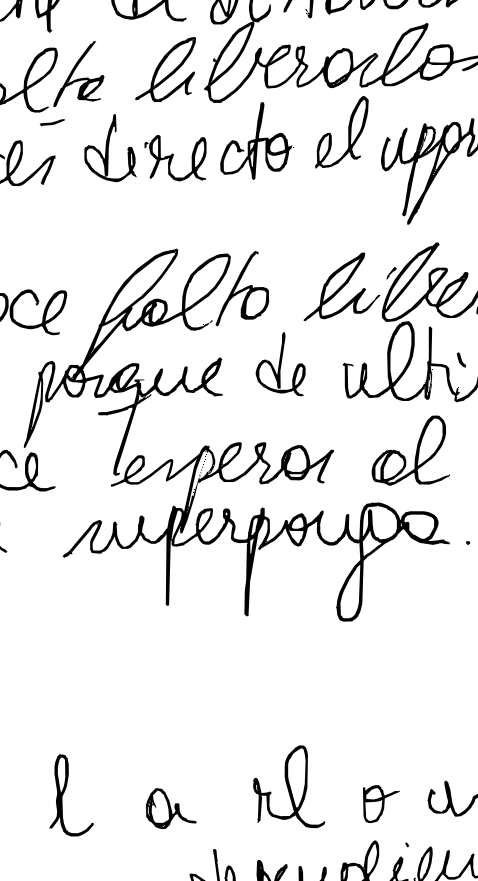
si me olvido de los "errores"
Historia serial equiv: $T_3; T_1; T_2$

b)

Schedule legal: no se pide lock sin que haya un lock del anterior
No serializable: hay conflictos (recursivos un ciclo).
ESTA MODO RARO con errores que decir o no.

CREO que se puede ni luego

... $rl_1(A); \dots u_2(A); \dots u_2(C); \dots rl_1(C)$



c)

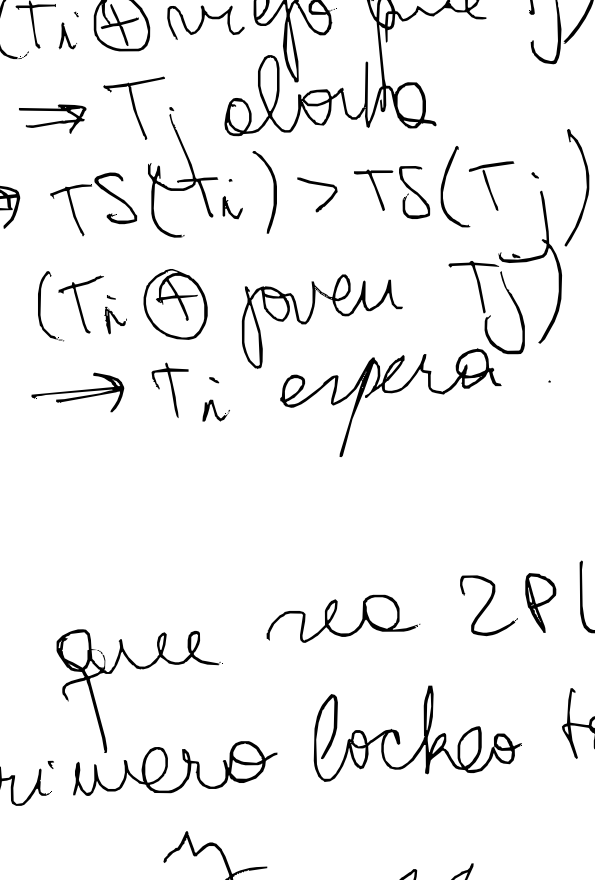
como el dirty real depende del commit, se podría identificar dirty real en $u_2(B); \dots rl_1(B)$
 T_1 lee de T_3 sin commit

3.6)

a)

H_1 Legal? Si
Toda la T_i 2PL? NO, T_1 no

b)



SR $\checkmark$ $\exists!$ historia serial equiv: $T_3; T_2; T_4; T_1$

c)

RC $\checkmark$

ACA $\times$

ST $\times$

d)

RC $\times$

ACA $\times$

ST $\times$

e)

uff que popo

3.7)

$H' = rl_3(A); rl_3(B); u_3(A); u_3(B); c_3$
 $rl_2(A); u_2(A); u_2(A); c_2$
 $rl_1(B); u_1(B); u_1(A); u_1(A); c_1$

$H = rl_3(A); u_3(A); rl_2(A); u_2(A); u_2(A);$
 $u_1(A); u_1(A);$
 $rl_3(B); u_3(B); rl_1(B); u_1(B); c_3; c_2; c_1$

RC, ACA, ~~2PL~~, Legal

c)

	S2PL	R2PL
T_1	$rl_1(B); u_1(A); u_1(A); u_1(B)$	$rl_1(B); u_1(A); c_1; u_1(A); u_1(B)$
T_2		
T_3		

NO SE PUEDE

3.8)

a) serializable no serial

$H = rl_1(A); r_1(A); rl_2(A); r_2(A);$ todo lo otro...

b)

$H' = u_1(A); r_1(A); u_2(A); \dots$
rechoyos

3.9)

a) di unamos lock tenemos ni un ul $\Rightarrow$ podemos producir Deadlock de la siguiente manera

$H = rl_1(A); r_1(A); rl_2(A); r_2(A);$
 $u_1(A); u_2(A)$

$\uparrow$
DEADLOCK...

(no pensamos como deadlock de sistemas operativos, sino dep de lo cotidiano)

b) $H' = u_1(A); r_1(A); u_2(A); r_2(A); u_1(A); u_2(A)$
 $\uparrow$
lo desapeando

(según diapo 54 de teoría)

3.10)

a) uno puede rl o ul a c/ operación y liberarlo directo, que no se puede lockado
b) a los que no pudiesen conflictos de DEADLOCK no hace falta liberarlos y le hace directo el upgrade/ul
c) no hace falta liberar todos, porque de ultimo lo hace esperar al ul que se superpone.

3.11)

a) EZ. l a rl o ul dependiendo de que haga

b)

$H = rl_1(A); r_1(A);$

FALTA HACER COMPLETO!

c)

probo wait-for
un nodo $x T_x$ y
un eje de T_i a T_j c/ $i \neq j$
ni eno lockado por T_j y
 T_i tiene que esperar p/ lockeo

3.12)

a) pl que no 2PL
primero lockeo todo
y
después libero todo

b)

RAJA

c, d)

idem 3.11.

3.13)

COMO DEFINEN EL DEADLOCK?

PORQUE $w_1(x); u_2(x)$ es deadlock?

NO ES QUE ESPERA A QUE SE LIBERE?

d)

T_i quiere lockeo x y T_j ya lo tiene:

• Wait-Die

$\Rightarrow TS(T_i) < TS(T_j)$
(T_i mas viejo que T_j)
 $\Rightarrow T_i$ espera

$\Rightarrow TS(T_i) > TS(T_j)$
(T_i mas joven que T_j)
 $\Rightarrow T_i$ aborta y recomienza

$\oplus$ force e/ultimo TS

• Wound-Wait

$\Rightarrow TS(T_i) < TS(T_j)$
($T_i \oplus$ viejo que T_j)
 $\Rightarrow T_j$ aborta

$\Rightarrow TS(T_i) > TS(T_j)$
($T_i \oplus$ joven que T_j)
 $\Rightarrow T_i$ espera

4) 1)

a) interleaving

t	T1	T2
1	st1=100	
2		st2=200
3	r1(A)	
4		r2(B)
5	w2(A)	
6		w1(B)

RT(A) = 100
RT(B) = 200
TS(T2) = 200 > RT(A) = 100
TS(T1) = 100 < RT(B) = 200
WRITE TOO LATE
∴ Rollback T1

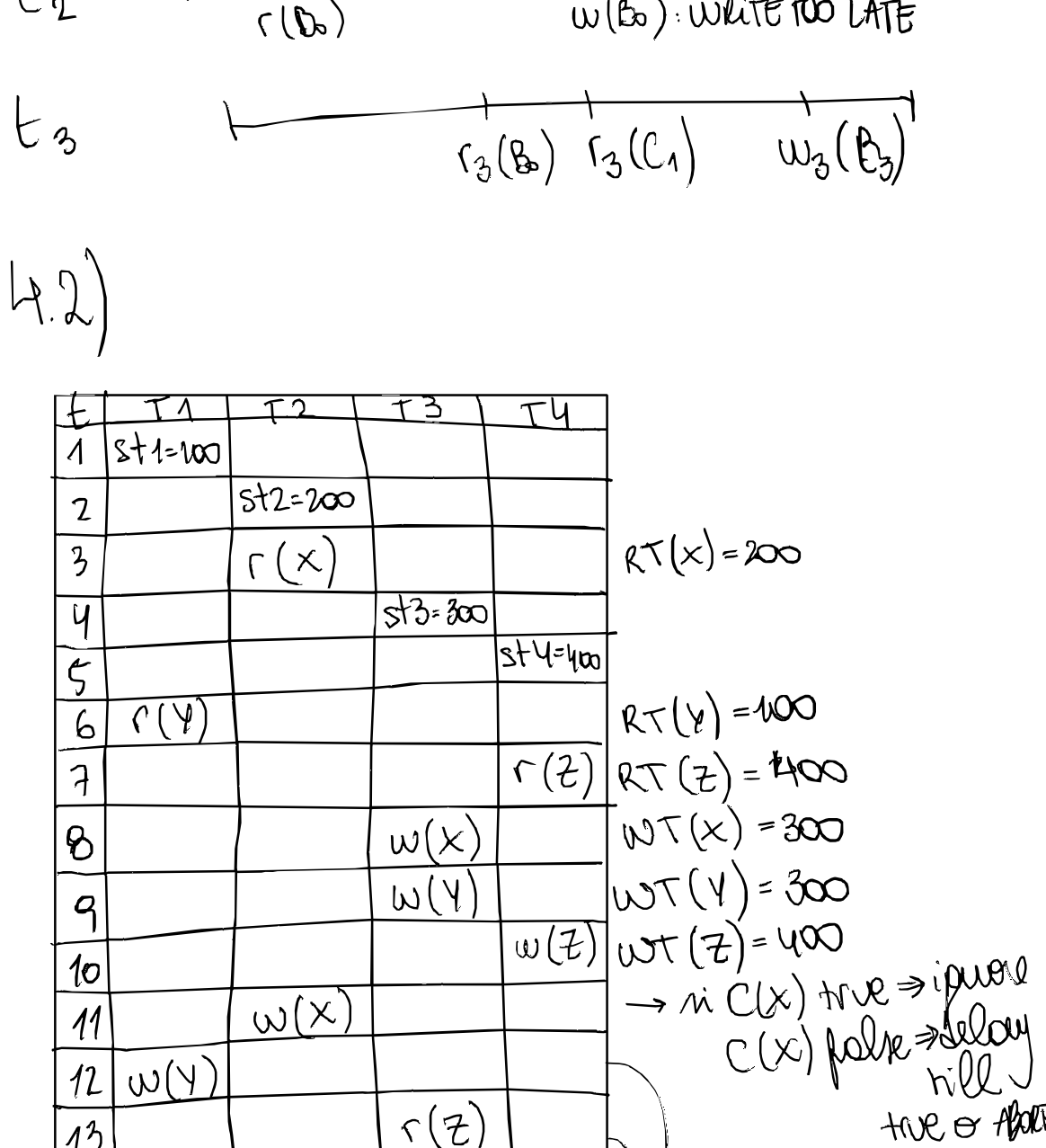
t	T1	T2
1	st1=100	
2	r1(A)	
3		st2=200
4		r2(B)
5		r2(A)
6	w1(B)	

RT(A) = 100
RT(B) = 200
RT(A) = 200
TS(T1) = 100 < RT(B) = 200
WRITE TOO LATE

t	T1	T2	T3
1	st1=100		
2		st2=200	
3			st3=300
4	r1(A)		
5		r2(B)	
6	w1(C)		
7			r3(B)
8			r3(C)
9		w2(B)	
10			w3(B)

RT(A) = 100
RT(B) = 200
WT(C) = 100
RT(B) = 300
RT(C) = 300
TS(T2) = 200 < RT(B) = 300
WRITE TOO LATE
∴ Rollback T2
→ WT(B) = 300

b) Multiversion MVTO



4.2)

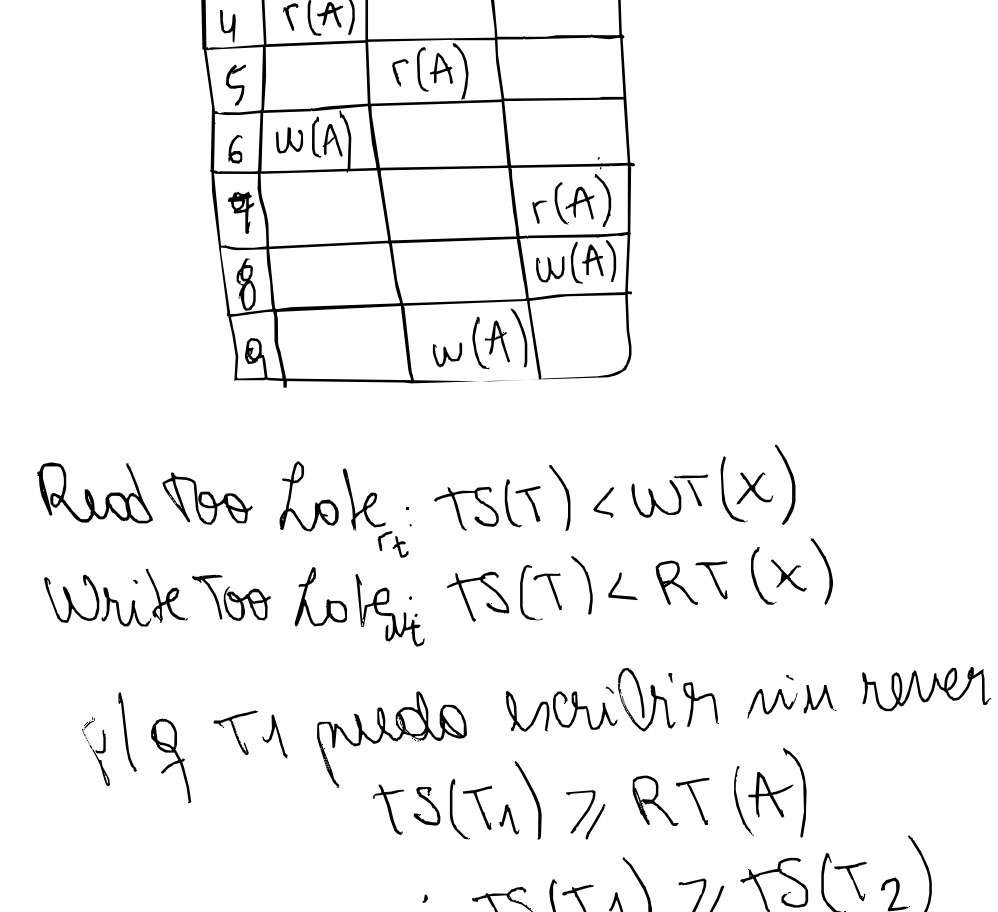
t	T1	T2	T3	T4
1	st1=100			
2		st2=200		
3		r(x)		
4			st3=300	
5				st4=400
6	r(y)			
7				r(z)
8			w(x)	
9			w(y)	
10				w(z)
11		w(x)		
12	w(y)			
13			r(z)	

RT(x) = 200
RT(y) = 100
RT(z) = 400
WT(x) = 300
WT(y) = 300
WT(z) = 400
→ si C(x) true ⇒ ignore
C(x) false ⇒ delay
true o abort
(Anuncio que C(x) true ⇒ ignore)

idem anterior TS(T1) = 100 > RT(y) = 100
TS(T1) = 100 < WT(y) = 300
∴ como ambos C(x) true ⇒ ignore
TS(T3) = 300 < WT(z) = 400 ∴ irreconciliable ⇒ abort

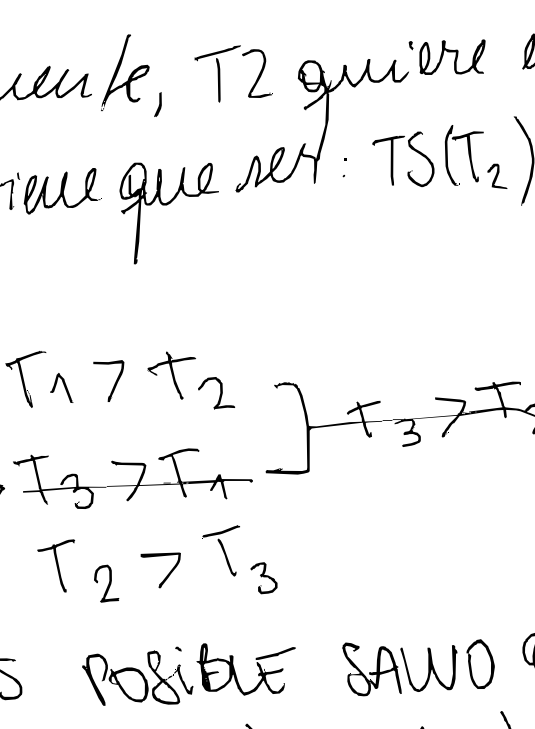
⇒ Como se aborta T3,
T1 y T2 terminan logrando escribir
∴ x = 2, y = 1, z = 4

b) MVTO



CREO que a priori de que t3 no aborta, x e y son anteriores a t2 y t1.

c)



oculto ⇒ MCSR

4.3) H = st1, st2, st3, r3(B); r2(C); r2(B); r2(A); w2(A); w2(B)

Aborta T2 porque hace w2(B) pero r3(B0) lee B en T3.

4.4)

a)

t	T1	T2	T3
1			st3=
2		st2=	
3	st1=		
4	r(A)		
5		r(A)	
6	w(A)		
7			r(A)
8			w(A)
9		w(A)	

Read Too Late: TS(T) < WT(x)

Write Too Late: TS(T) < RT(x)

p/q T1 pueda escribir sin revert
TS(T1) > RT(A)
∴ TS(T1) > TS(T2)

si T3 quiere leer ⇒ TS(T3) > WT(A)
y el ultimo WT(A) fue TS(T1)
⇒ TS(T3) > TS(T1)

puede escribir? TS(T3) > RT(A) = TS(T3)

y finalmente, T2 quiere escribir
pero tiene que ser: TS(T2) > RT(A)
TS(T3)

⇒ T1 > T2
T3 > T1
T2 > T3

NO ES POSIBLE SABO QUE
muestra, podria hacer que T3 no arranque
hasta que T1 no haya concluido

TS(T2) = TS(T3)

TS(T2) = TS(T3)

TS(T2) = TS(T3)

TS(T2) = TS(T3)

TS(T2) = TS(T3)

TS(T2) = TS(T3)

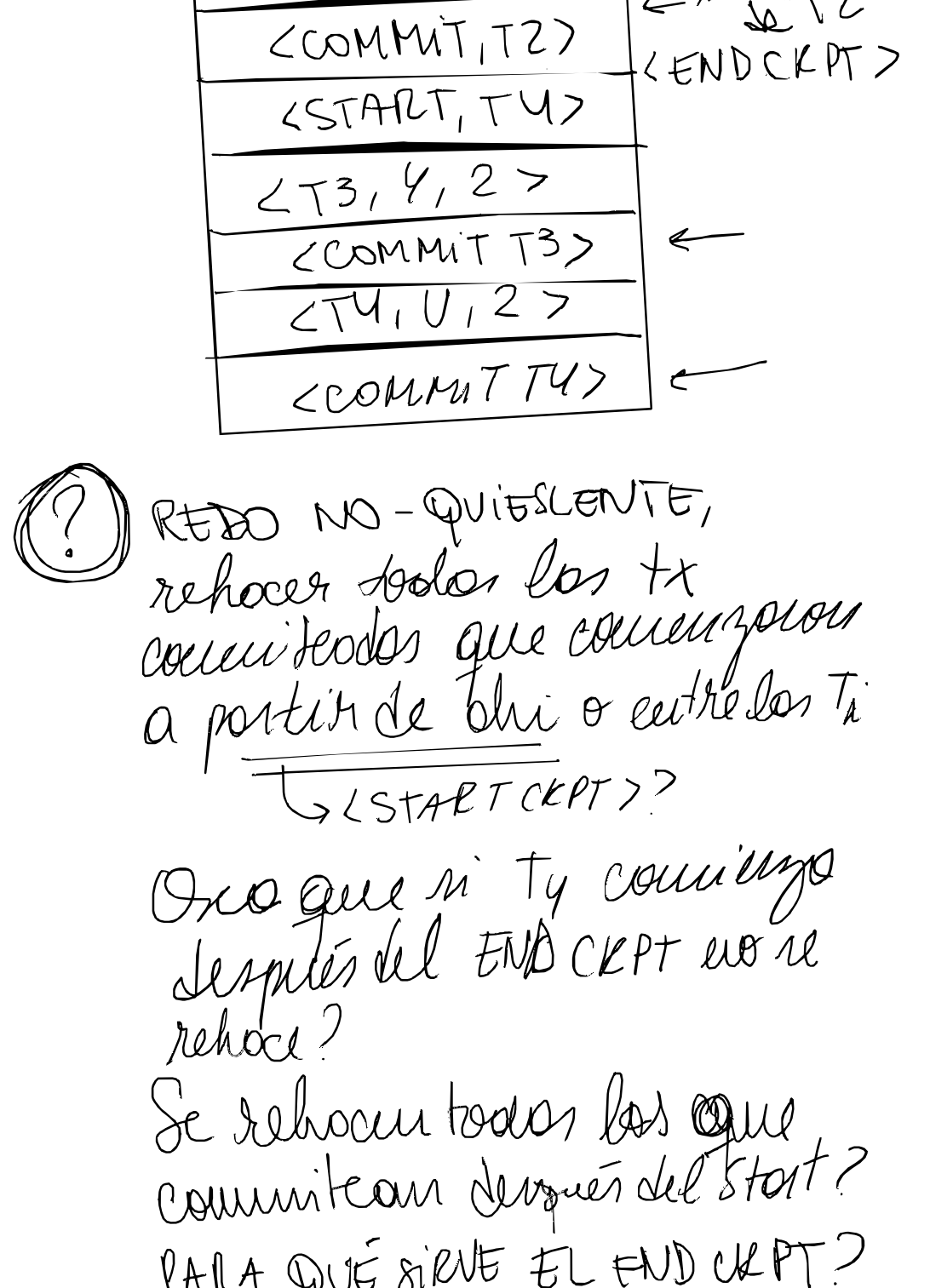
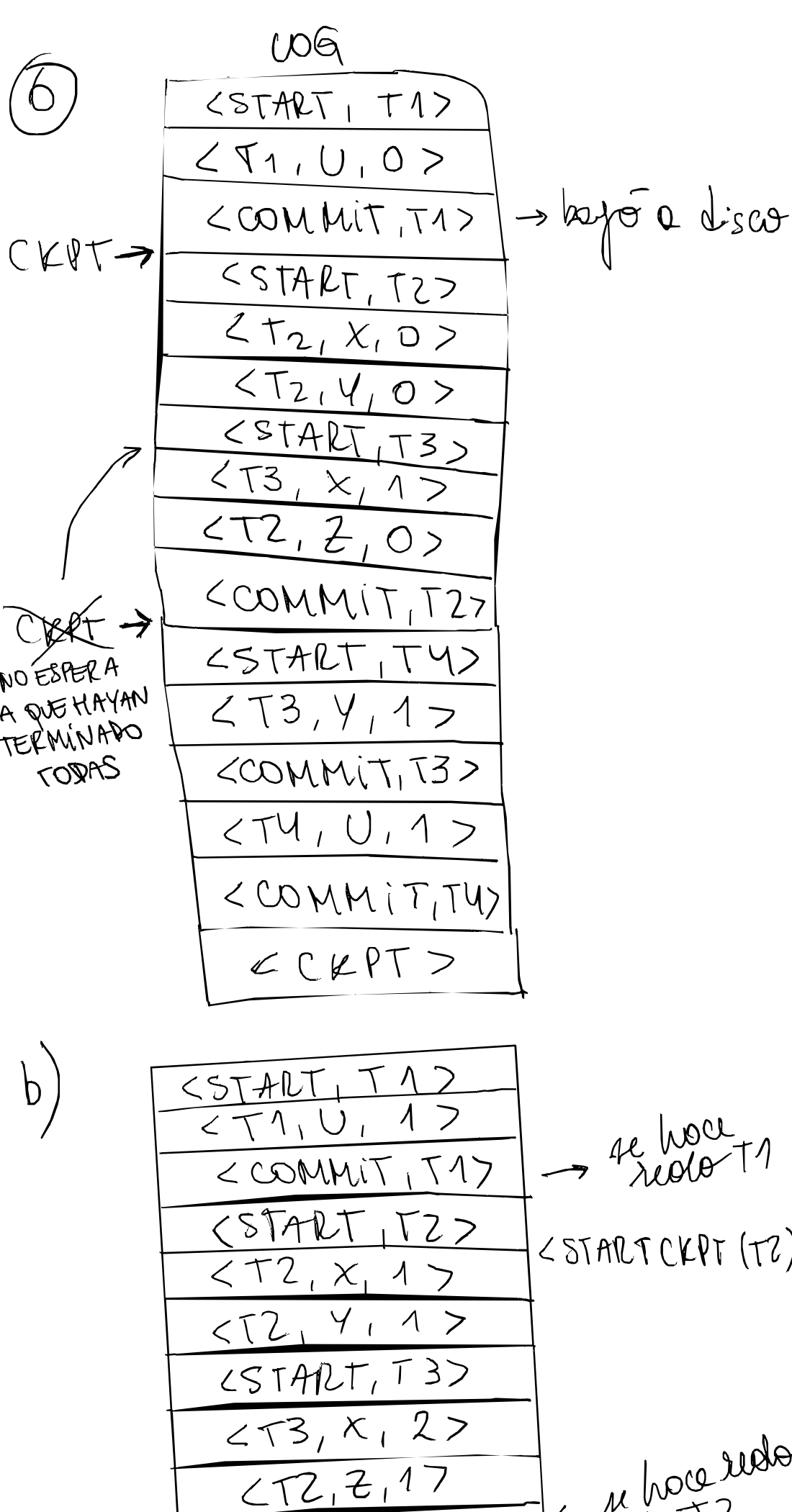
TS(T2) = TS(T3)

TS(T2) = TS(T3)

TS(T2) = TS(T3)

⑤ WTF Tx SQL?

X



¿? REDO NO-QUIESCENTE, rehacer todos los tx cometidos que comenzaron a partir de bhi o entre las T_i

¿<START CKPT>?

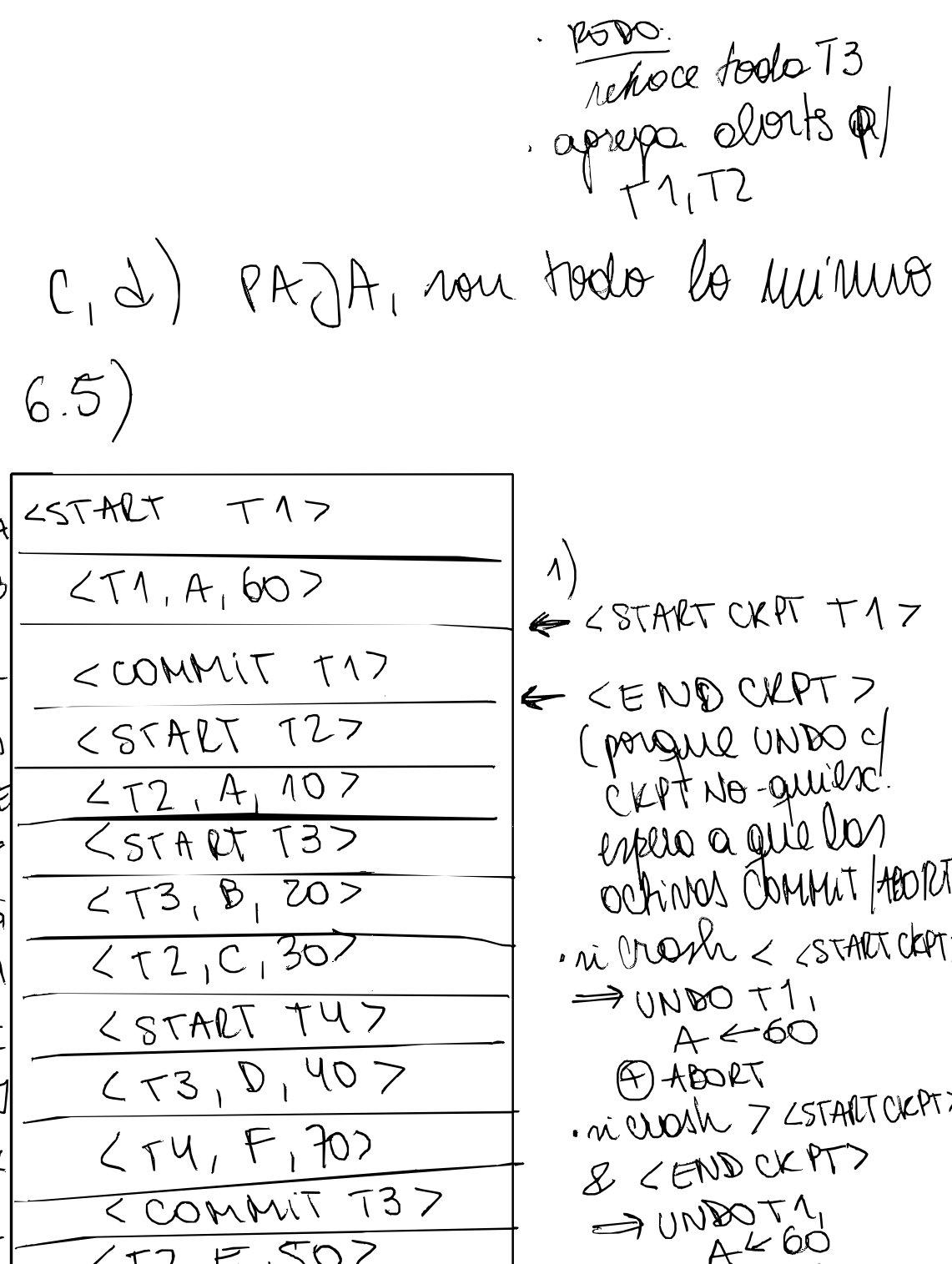
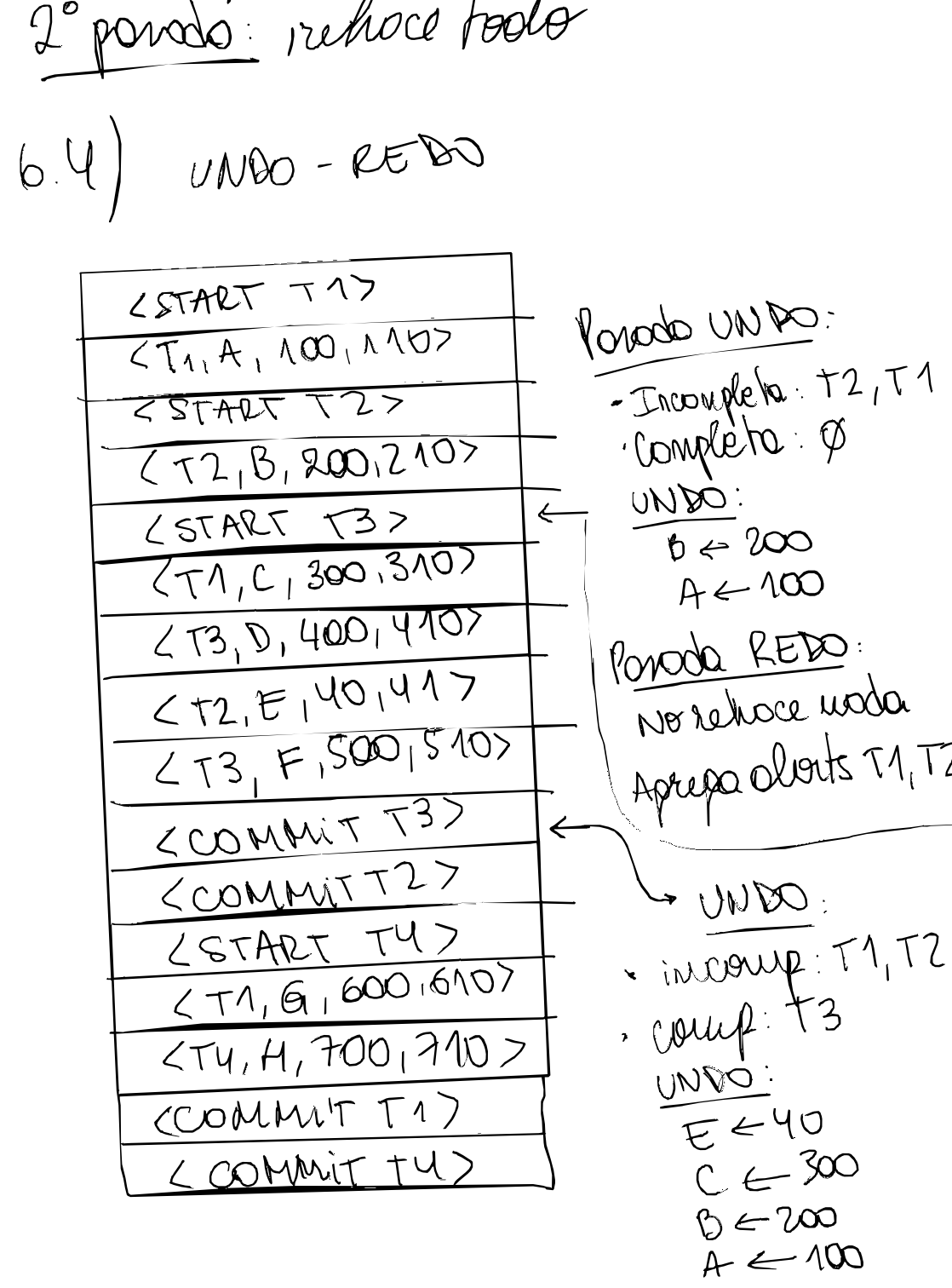
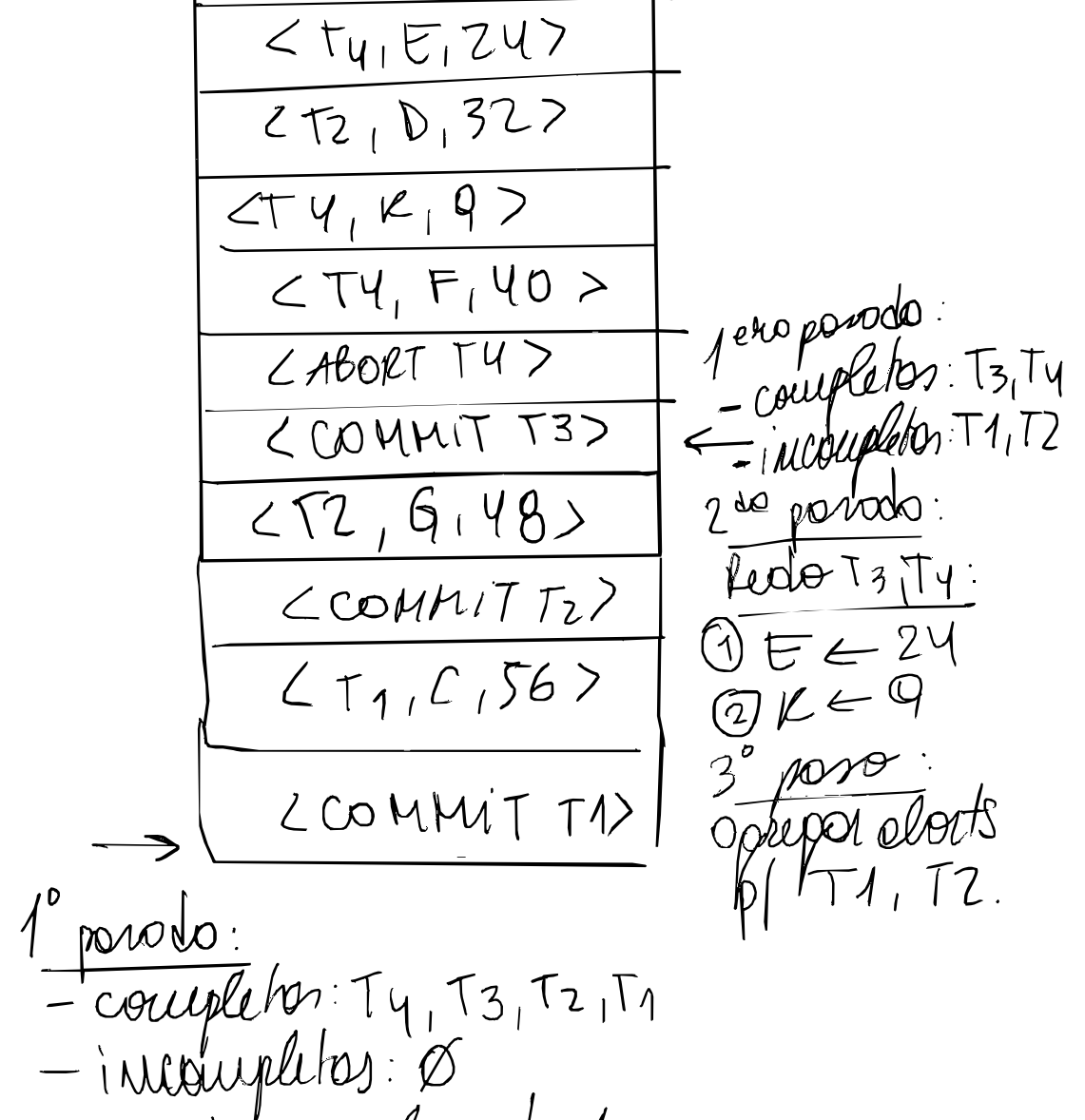
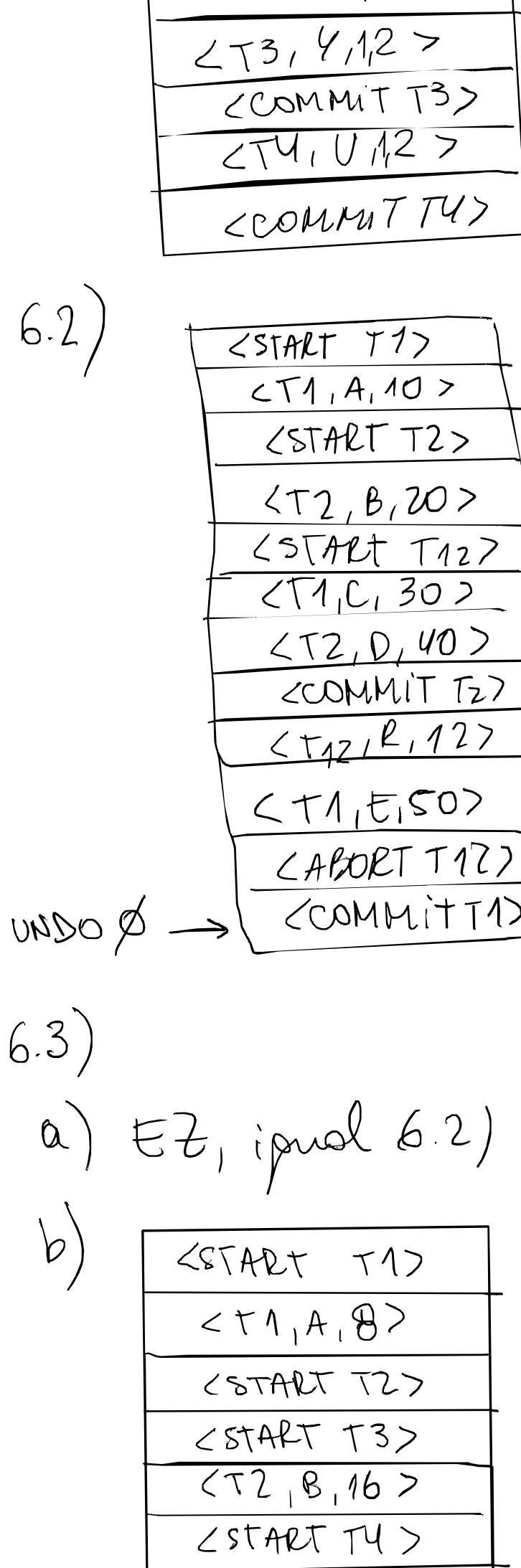
Ojo que si T_4 comienza después del END CKPT no se rehace?

Se rehacen todos los que comitean después del start?

PARA QUÉ sirve EL END CKPT?

→ COMMIT de Redo logging se hace antes de hacer todos al disco

⇒ ver un start CKPT me dice que todos los cometidos previos se hizo flush ⇒ esto en disco, y solo después que se haga de ahí en adelante.



c, d) PAJA, no todo lo mismo

6.5)

<START T1>
<T1, A, 60>
<COMMIT T1>
<START T2>
<T2, A, 10>
<START T3>
<T3, B, 20>
<T2, C, 30>
<START T4>
<T3, D, 40>
<T4, F, 70>
<COMMIT T3>
<T2, E, 50>
<COMMIT T2>
<T4, B, 80>
<COMMIT T4>

1) <START CKPT T1>

<END CKPT> (porque UNDO CKPT no quiere esperar a que los otros Commit/Abort)

• si crash < <START CKPT> ⇒ UNDO T1, A ← 60
④ ABORT

• si crash > <START CKPT> & <END CKPT> ⇒ UNDO T1, A ← 60
④ ABORT

• si crash > <END CKPT> ⇒ NO UNDO ni ABORT

¿? POOL DE BUFFERS ≠ DISCO

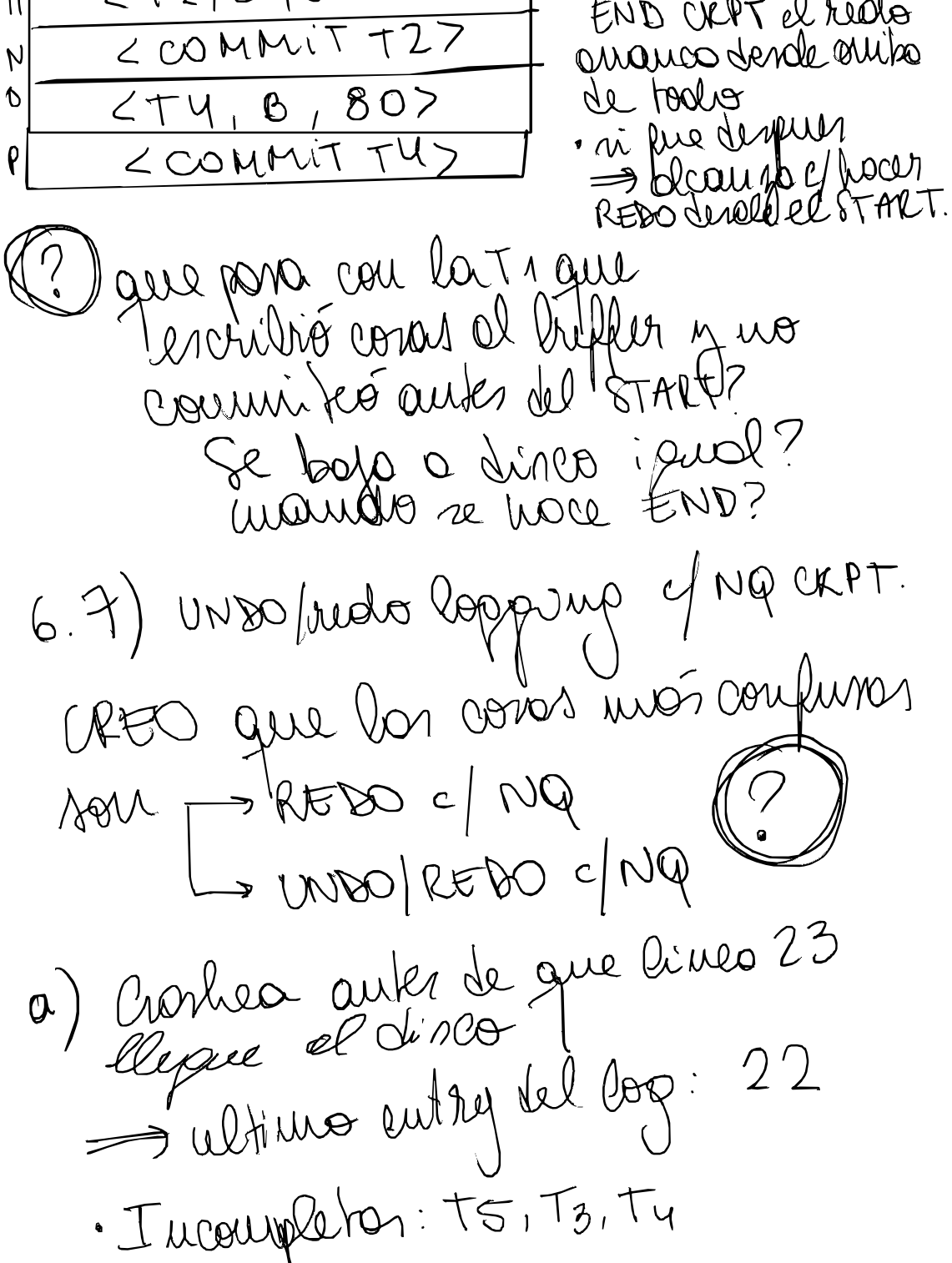
2) E-F: <START CKPT T2>
N-O: <END CKPT>

• crash < <START CKPT>: UNDO T2: A ← 10

• crash > <START CKPT> & <END CKPT>: UNDO HASTA ④

• crash > <END CKPT>: UNDO T4; B ← 80

3, 4, 5) PAJA, se larga lo haces todo



¿? que pasa con la T1 que escribió cosas al buffer y no comiteó antes del START?

Se baja a disco igual? cuando se hace END?

6.7) **UNDO/redo logging y NQ CKPT.**

CKPT que los cosas más confusas son

→ REDO c/ NQ

→ UNDO/REDO c/ NQ

¿?

a) Crasha antes de que línea 23 llegue al disco

⇒ último entry del log: 22

• Incompletos: T5, T3, T4

• Completos: T2, T1

∴ Deshace los incompletos:

G ← 43 ← 27

E ← 35 ← 21

F ← 37 ← 19

④ ABORT p/c 1.

④ flush del log

Cómo hay END CKPT, solo hace REDO desde START CKPT en adelante

⇒ C ← 10

	A	B	C	D	E	F	G	H
a	20	24	10	4	21	19	27	?
b								
c								
d								

PAJA, PAJA, PAJA